# TutorRAG

AI-Powered Educational Assistant

with Retrieval-Augmented Generation

Generated: 2026-07-24

## Project Overview

TutorRAG is an AI-powered educational platform that uses Retrieval-Augmented Generation (RAG) to help students learn from their own uploaded documents. Teachers upload PDF textbooks or notes; the system extracts text, indexes it in a vector database, and allows students to ask questions and receive accurate answers grounded in the uploaded content with source citations.

The platform also supports automated quiz generation, allowing students to create multiple-choice tests on any topic from the available documents, submit answers, and receive instant grading with feedback.

## Key Features

- Document Upload - Teachers upload PDF textbooks/notes; the system extracts and indexes them.
- AI Q&A Chat - Students ask questions and get answers grounded in the uploaded content with source citations.
- Quiz Generation - Students generate multiple-choice quizzes on any topic from the documents.
- Quiz History - Review past quiz attempts with per-question feedback and scoring.
- Grade-Level Filtering - Content is indexed by grade level for age-appropriate responses.

## Architecture

The application follows a client-server architecture with a Python FastAPI backend and a Streamlit frontend. The backend orchestrates document processing, vector storage, retrieval, and LLM inference.

### Data Flow (Student Question)

1. Student types a question in the Streamlit chat interface.
2. Client sends POST /chat with HTTP Basic Auth.
3. Server authenticates against MongoDB users collection.
4. Question is embedded using Google gemini-embedding-001.
5. Pinecone returns top-5 most similar document chunks.
6. Full chunk text retrieved from MongoDB, ranked by relevance.
7. Context + question sent to Groq Llama 3.3 70B for answer.
8. Answer and source filenames returned to the client.
9. Q&A pair logged in MongoDB chat_history.

## Tech Stack

| Component | Technology | Role |
|---|---|---|
| Backend | FastAPI | REST API server |
| Frontend | Streamlit | Web UI |
| LLM | Groq (Llama 3.3 70B) | Answer generation |
| Embeddings | Google Gemini Embedding 001 | Vector embeddings |
| Vector DB | Pinecone | Similarity search |
| Document DB | MongoDB Atlas | User & content storage |
| Auth | HTTP Basic + bcrypt | Authentication |
| PDF Parsing | LangChain PyPDFLoader | Text extraction |

## Project Structure

```
server/              # FastAPI backend
  main.py            # App entry point
  config/db.py       # MongoDB connection
  auth/              # Signup, login, hashing
  docs/              # PDF upload & vector store
  chat/              # Q&A chat & quiz logic
client/              # Streamlit frontend
  main.py            # Multi-page UI (landing, login, signup, dashboards)
```

## Setup Instructions

### Prerequisites

Python 3.10+, MongoDB Atlas instance, Pinecone account with 'tutor-rags' index, Google Generative AI API key, Groq API key.

### Installation

```
python -m venv .venv
.venv\Scripts\activate
pip install -r server/requirements.txt
```

### Configuration

Edit server/.env with your API keys:

```
MONGO_URI=mongodb+srv://...
DB_NAME=tutorRag
PINECONE_INDEX_NAME=tutor-rags
GOOGLE_API_KEY=...
PINECONE_API_KEY=...
GROQ_API_KEY=...
```

### Run

```
# Terminal 1 - Backend
uvicorn server.main:app --reload --port 8000

# Terminal 2 - Frontend
pip install -r client/requirements.txt
streamlit run client/main.py
```

## API Endpoints

| Method | Path | Auth | Description |
|---|---|---|---|
| POST | /signup/student | No | Register student |
| POST | /signup/teacher | No | Register teacher |
| GET | /login | Basic | Authenticate, return role |
| POST | /upload_docs | Basic | Upload PDF (Teacher only) |
| POST | /chat | Basic | Ask question (Student) |
| POST | /quiz | Basic | Generate quiz (Student) |
| POST | /quiz/check | Basic | Submit quiz answers (Student) |
| GET | /quiz/history | Basic | Quiz attempt history (Student) |

## User Roles

### Teacher

Upload PDF documents and specify a target grade level. Content is indexed and made available to students.

### Student

Ask questions with source citations, generate and take quizzes, review quiz history with per-question feedback.

## RAG Pipeline Detail

The core of TutorRAG is the Retrieval-Augmented Generation pipeline:

1. Document Ingestion: PDF text is extracted using PyPDFLoader, split into 500-char chunks (50 overlap) using RecursiveCharacterTextSplitter, stored in MongoDB with metadata (doc_id, page, source, grade, role).

2. Embedding: Each chunk is embedded via Google gemini-embedding-001 and upserted into Pinecone index tutor-rags with the same metadata.

3. Retrieval: Student question is embedded, Pinecone top-5 similarity search with grade/role filters. Full chunks fetched from MongoDB, ranked by relevance.

4. Generation: Context + question sent to Groq Llama 3.3 70B via a RAG prompt. Answer and source filenames returned.

## Notes

- Auth uses HTTP Basic Auth with bcrypt-hashed passwords in MongoDB.
- Ensure server/.env with API keys is excluded from version control.
- Content access controlled via metadata filters (role, grade) on Pinecone queries.
- The Streamlit client runs on port 8501 by default; the backend runs on port 8000.

*Generated from project source files.*